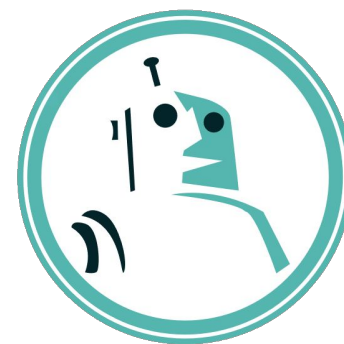


Sistemas Operacionais

PROCESSOS E THREADS NO LINUX E MACOS

Bacharelado em Ciência da Computação

Prof. Filipo Novo Mór



AGENDA DA AULA

Bloco 1: Processos

- Conceito e Memória
- O PCB (Process Control Block)
- Criação via syscall fork()

Bloco 2: Threads

- Modelo Lightweight Process
- O padrão POSIX (pthreads)
- Comunicação e Mutex

01. Processos

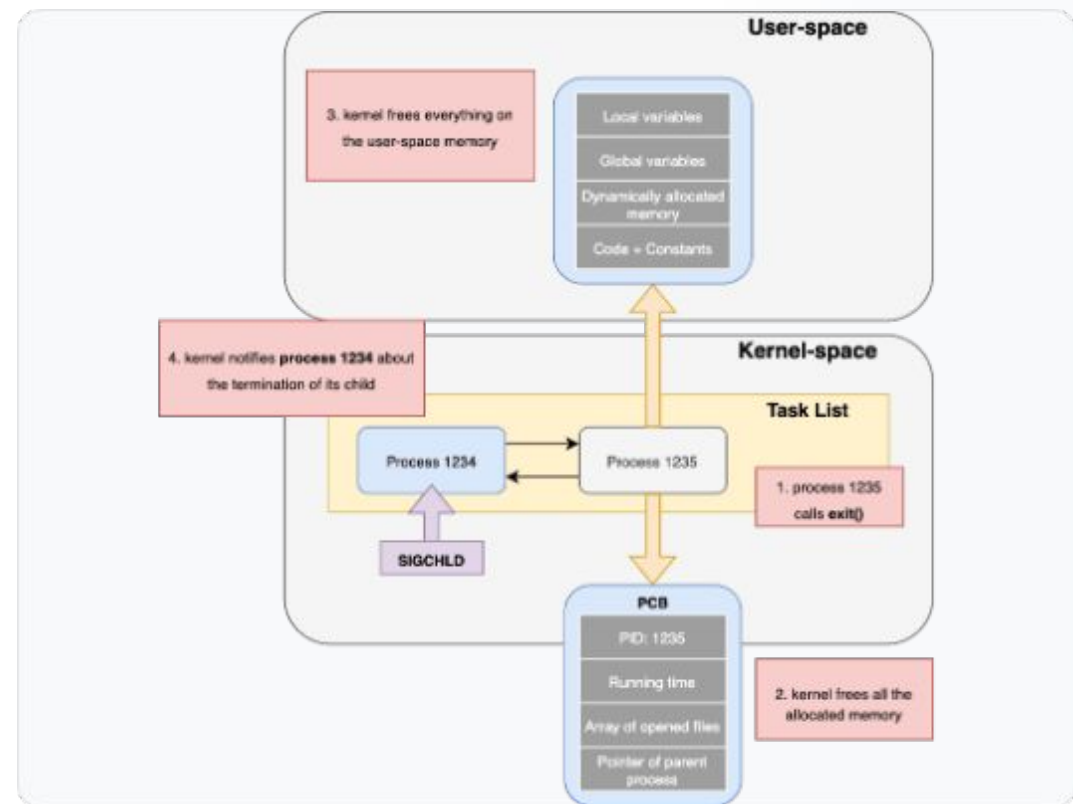
A unidade fundamental de isolamento

DEFINIÇÃO DE PROCESSO

Um processo é um **programa em execução**.
Diferente do código estático em disco, ele é uma entidade dinâmica.

Composição:

- **Código (Text):** Instruções binárias.
- **Dados:** Variáveis globais e estáticas.
- **Pilha (Stack):** Funções e variáveis locais.
- **Heap:** Memória alocada dinamicamente.



| CRIAÇÃO: SYSCALL FORK()

O Mecanismo

No Linux/macOS, processos são criados clonando um processo pai. O filho herda uma cópia do espaço de memória.

O Retorno

- **Para o Pai:** Retorna o PID do filho.
- **Para o Filho:** Retorna 0.
- **Erro:** Retorna -1.

EXEMPLO C: FORK()

```
#include
#include

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Filho executando\n");
    } else {
        printf("Pai criou o processo %d\n", pid);
    }
    return 0;
}
```

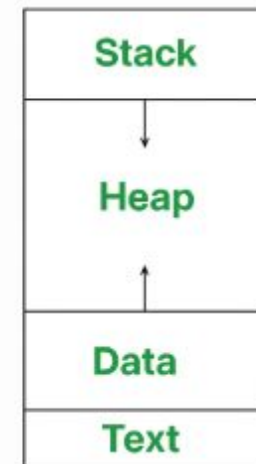
TRAÇADO DE EXECUÇÃO: FORK()

Fluxo de Memória

No momento do `fork()`, o SO duplica o processo. Agora existem dois **Program Counters** apontando para a próxima linha.

Terminal Output:

```
$ gcc programa.c -o exec
$ ./exec
Pai criou o processo 4502
Filho executando
```



02. Threads

Concorrência dentro do mesmo espaço

CONCEITO DE THREAD

"Lightweight Process"

Uma thread é um fluxo de execução dentro de um processo. Enquanto processos são isolados, threads **compartilham** a mesma memória.

O que é compartilhado?

- Segmento de Código e Dados (Globais).
- Arquivos abertos e Sinais.
- **Privado:** Cada thread tem seu Stack e Registradores.

| CRIAÇÃO: PTHREADS

No Linux e macOS, usamos a biblioteca **POSIX Threads**. A criação não duplica o processo, apenas inicia uma nova função paralelamente.

```
pthread_create(&thread_id, NULL, funcao, NULL);
```

Diferente de processos, threads exigem que o processo principal espere por elas (pthread_join).

EXEMPLO C: PTHREADS

```
void* print_msg(void* ptr) {  
    printf("Executando na Thread!\n" );  
    return NULL;  
}  
  
int main() {  
    pthread_t thread;  
    pthread_create(&thread, NULL, print_msg, NULL);  
    pthread_join(thread, NULL); // Espera a thread  
    printf("Main finalizado\n" );  
    return 0;  
}
```

TRAÇADO DE EXECUÇÃO: THREAD

Observação Crucial

Se o main terminar sem o pthread_join, o processo morre e todas as threads são encerradas abruptamente.

Terminal Output:

```
$ gcc t.c -lpthread -o exec  
$ ./exec  
Executando na Thread!  
Main finalizado
```

Custo: Criar uma thread é até 100x mais rápido que um processo, pois não há alocação de nova tabela de páginas de memória.

PROCESSO VS. THREAD

Característica	Processo	Thread
Memória	Isolada (Privada)	Compartilhada (Mesmo Processo)
Troca de Contexto	Alta (Custo do SO)	Baixa (Leve)
Falhas	Isoladas (Um não afeta outro)	Críticas (Um erro derruba todas)
Comunicação	Difícil (IPC via Kernel)	Fácil (Variáveis Globais)

IPC: INTER-PROCESS COMMUNICATION

Como processos não acessam a memória uns dos outros, o Kernel atua como mediador através de **IPC**.



Pipes

Fluxo unidirecional de bytes.



Shared Mem

Área de memória mapeada
em ambos.



Messages

Fila de mensagens via Kernel.

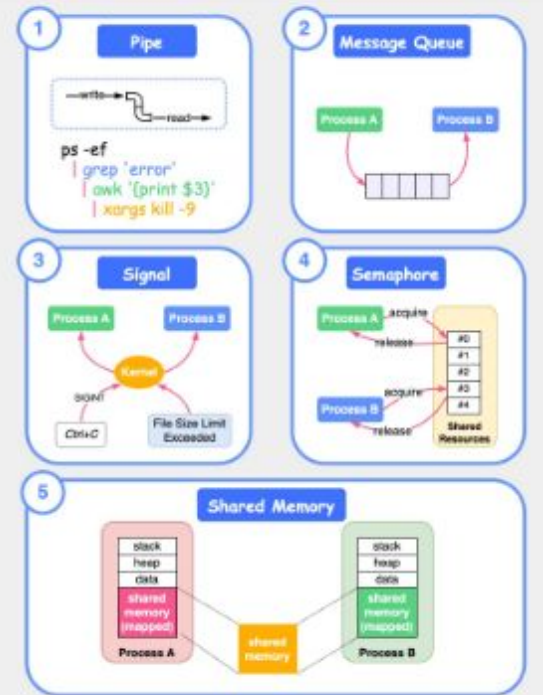
EXEMPLO IPC: PIPES

```
int fd[2]; pipe(fd);
if (fork() == 0) {
    write(fd[1], "OLA", 3);
} else {
    read(fd[0], buf, 3);
    printf("Recebido: %s\n", buf);
}
```

Terminal Output:

Recebido: OLA

5 Inter-Process Communications blog.bytebytego.com



ITC: INTER-THREAD COMMUNICATION

Threads conversam via variáveis globais. **Problema:** Condições de Corrida (Race Conditions).

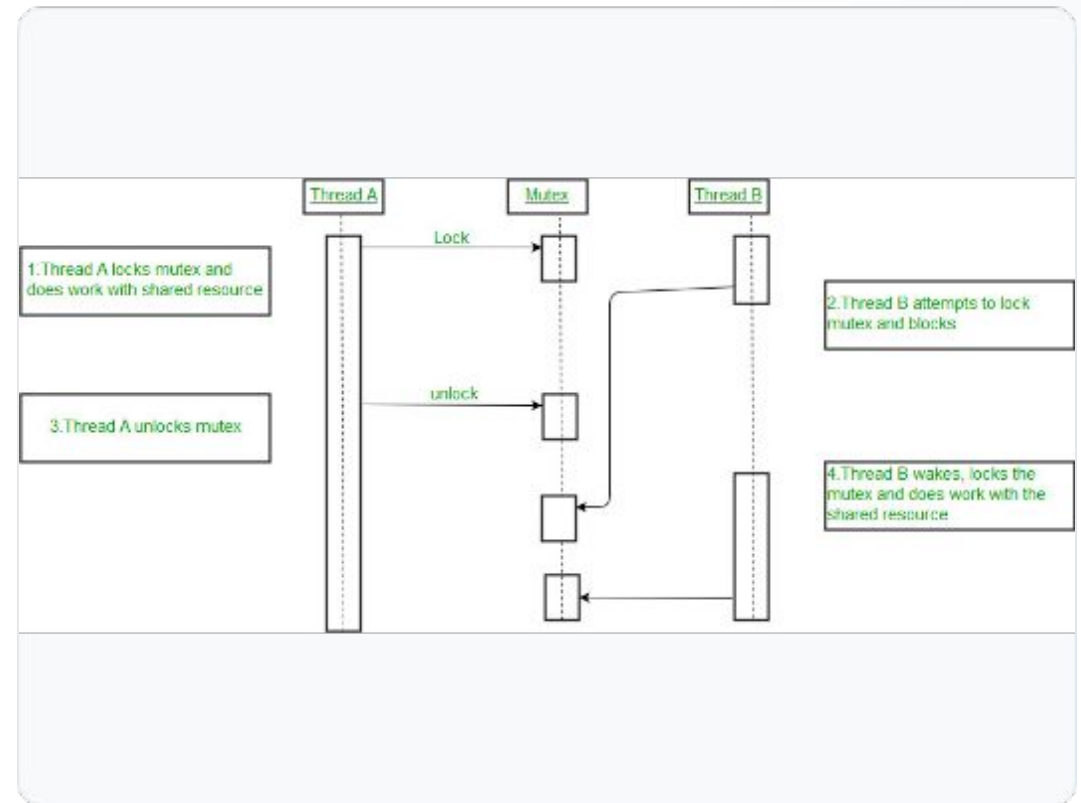
O Perigo

Se duas threads somarem 1 em $x = 0$ simultaneamente, o resultado pode ser 1 em vez de 2, devido à intercalação das instruções da CPU.

| A SOLUÇÃO: MUTEX

O **Mutex** (Mutual Exclusion) garante que apenas uma thread entre na "Região Crítica" por vez.

```
pthread_mutex_lock(&m);  
saldo = saldo + 100;  
pthread_mutex_unlock(&m);
```



RASTREIO: SINCRONIZAÇÃO

Sem Mutex

Thread 1 lê x=10

Thread 2 lê x=10

Thread 1 escreve x=11

Thread 2 escreve x=11

Resultado Errado: 11

Com Mutex

Thread 1 tranca Mutex

Thread 1 lê/escreve x=11

Thread 1 destranca Mutex

Thread 2 tranca... escreve x=12

Resultado Correto: 12

RESUMO FINAL

Processos

Segurança máxima, isolamento, custo alto, comunicação via Kernel.

Threads

Velocidade, compartilhamento, risco de corrupção de dados, comunicação via memória.

"Use processos para segurança e threads para performance."

Dúvidas?

Sistemas Operacionais | Ciência da Computação

Bacharelado em Ciência da Computação

Prof. Filipo Novo Mór

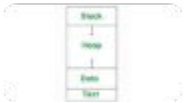


IMAGE SOURCES



<https://www.baeldung.com/wp-content/uploads/sites/2/2023/10/Process-Control-Block-Structure.png>

Source: www.baeldung.com



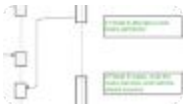
https://media.geeksforgeeks.org/wp-content/uploads/20251224114920150586/Process_look.png

Source: www.geeksforgeeks.org



<https://assets.bytebytego.com/diagrams/0234-inter-process-communication.png>

Source: bytebytego.com



https://media.geeksforgeeks.org/wp-content/uploads/Mutex_lock_for_linux.jpg

Source: www.geeksforgeeks.org